SOBRECARGA DE OPERADORES

Prof. André Backes

Sobrecarga de operadores

- Permite definir o comportamento de alguns operadores básicos da linguagem para tipos de dados personalizados, como as classes
- Exemplo de operadores
  - ==, >, <, +, -, *, etc.

Sobrecarga de operadores

- É apenas uma conveniência
  - Torna as operações mais intuitivas e facilitando o seu uso
- Com a sobrecarga, podemos escrever
  - **`p1 + p2`** ao invés de **`p1.soma(p2)`**
  - **`p1 == p2`** ao invés de **`p1.igual(p2)`**

Sobrecarga de operadores

- Cada operador é representado por um método especial
- Alguns deles

Operador	Método
Adição (+)	operator+
Subtração (-)	operator-
Multiplicação (*)	operator*
Divisão (/)	operator/
Módulo (%)	operator%
Igualdade (==)	operator==
Desigualdade (!=)	operator!=
Maior que (>)	operator>
Menor que (<)	operator<
Maior ou igual que (>=)	operator>=
Menor ou igual que (<=)	operator<=
E lógico (&&)	operator&&
Ou lógico (||)	operator||

Somando dois objetos

- Pode ser feita com uma função simples
  - Números complexos, partes reais e imaginárias
  - Soma parte real com parte real, e a parte imaginária com a parte imaginária
  - Retorna um novo objeto

```
class Complexo {
private:
    double real;
    double imag;
public:
    Complexo(double r, double i): real {r}, imag {i}{ }
    // Método para a adição
    Complexo soma(const Complexo& outro) const {
        return Complexo(real + outro.real, imag + outro.imag);
    }

    void mostrar() const {
        cout << real << " + " << imag << "i" << endl;
    }
};

int main() {
    Complexo c1(2, 3);
    Complexo c2(4, 5);

    Complexo resultado = c1.soma(c2);
    resultado.mostrar();

    return 0;
}
```

Somando dois objetos

- Método especial **operator+**
  - A essência do método permanece a mesma
  - Operação fica mais intuitiva
  - Código mais legível e mais próximo do que seria naturalmente esperado pelos programadores

```
class Complexo {
private:
    double real;
    double imag;
public:
    Complexo(double r, double i): real {r}, imag {i}{ }
    // Sobrecarga do operador de adição
    Complexo operator+(const Complexo& outro) const {
        return Complexo(real + outro.real, imag + outro.imag);
    }

    void mostrar() const {
        cout << real << " + " << imag << "i" << endl;
    }
};

int main() {
    Complexo c1(2, 3);
    Complexo c2(4, 5);

    Complexo resultado = c1 + c2;
    resultado.mostrar();

    return 0;
}
```

Comparando o conteúdo dos objetos

- Pode ser feita com uma função simples
  - Compara dois objetos diferentes representando coordenadas **(x,y)**
  - Comparação é pelo conteúdo
  - Retornar um valor do tipo **bool**

```
class Ponto {
private:
    int x;
    int y;
public:
    Ponto(int _x, int _y): x {_x}, y {_y}{ }

    bool igual(const Ponto& outro) const {
        return (x == outro.x && y == outro.y);
    }
};

int main() {
    Ponto p1(2, 3);
    Ponto p2(2, 3);

    if (p1.igual(p2)) {
        cout << "Os pontos sao iguais!" << endl;
    } else {
        cout << "Os pontos sao diferentes!" << endl;
    }

    return 0;
}
```

Comparando o conteúdo dos objetos

- Método especial **operator==**
  - Novamente, a essência do método permanece a mesma
  - Operação fica mais intuitiva

```
class Ponto {
private:
    int x;
    int y;
public:
    Ponto(int _x, int _y): x {_x}, y {_y}{ }

    // Sobrecarga do operador de comparação
    bool operator==(const Ponto& outro) const {
        return (x == outro.x && y == outro.y);
    }
};

int main() {
    Ponto p1(2, 3);
    Ponto p2(2, 3);

    if (p1 == p2) {
        cout << "Os pontos sao iguais!" << endl;
    } else {
        cout << "Os pontos sao diferentes!" << endl;
    }

    return 0;
}
```

Sobrecarga com função externa

- Não é necessário definir uma função dentro da classe
  - Podemos criar uma função externa a nossa classe
  - Deve ter o nome do operador a ser sobrecarregado
  - Deve aceitar como parâmetros os objetos da classe envolvidos na operação

Sobrecarga com função externa

- Criando uma função externa
  - Recebe dois objetos ao invés de um
    - Não temos mais o ponteiro **this** implícito
  - Método não pertence a classe
    - Não tem acesso aos atributos privados
    - Acesso é feito usando os métodos disponibilizados para acesso

```
class Inteiro{
private:
    int valor;

public:
    Inteiro(int _valor): valor {_valor}{ }

    int getInteiro() const { return valor; }
};

Inteiro operator+(const Inteiro& v1, const Inteiro& v2){
    return Inteiro(v1.getInteiro() + v2.getInteiro());
}

int main(){
    Inteiro inteiro1(6);
    Inteiro inteiro2(8);
    Inteiro inteiroSoma(inteiro1 + inteiro2);
    cout << "Valor: " << inteiroSoma.getInteiro() << endl;

    return 0;
}
```

Sobrecarga com função externa

- A palavra-chave **friend** permite conceder acesso especial a partes privadas de uma classe para funções ou outras classes específicas
  - Permite que a função ou classe **friend** acesse membros privados (ou protegidos) da classe em questão sem que precisemos recorrer a outras soluções, como a herança

Sobrecarga com função externa

- Simplificando o acesso com **friend**
  - Concede acesso especial a partes privadas da classe
  - O protótipo da função precisa ser inserida dentro da classe

```
class Inteiro{
private:
    int valor;

public:
    Inteiro(int _valor): valor {_valor}{ }
    // Declaração da sobrecarga de operador como "friend"
    friend Inteiro operator+(const Inteiro& v1, const Inteiro& v2);

    int getInteiro() const { return valor; }
};

Inteiro operator+(const Inteiro& v1, const Inteiro& v2){
    // A função "friend" acessa membros privados da classe
    return Inteiro(v1.valor + v2.valor);
}

int main(){
    Inteiro inteiro1(6);
    Inteiro inteiro2(8);
    Inteiro inteiroSoma = inteiro1 + inteiro2;
    cout << "Valor: " << inteiroSoma.getInteiro() << endl;

    return 0;
}
```

Sobrecarga com função externa

- **friend** é especialmente útil quando o operador precisa acessar membros privados de duas classes diferentes
  - Protótipo da classe **Real** inserido no começo
  - A função de sobrecarga é declarada como **friend** de ambas as classes

```
class Real; //protótipo da classe Real

class Inteiro{
private:
    int valor;
public:
    Inteiro(int _valor): valor {_valor}{ }
    friend Real operator+(const Inteiro& v1,const Real& v2);

    int getInteiro() const { return valor; }
};

class Real{
private:
    double valor;
public:
    Real(double _valor): valor {_valor}{ }
    friend Real operator+(const Inteiro& v1,const Real& v2);

    double getReal() const { return valor; }
};

Real operator+(const Inteiro& v1, const Real& v2){
    return Real(v1.valor + v2.valor);
}

int main(){
    Inteiro obj1(6);
    Real obj2(5.10);
    Real obj3 = obj1 + obj2;

    cout << "Valor: " << obj3.getReal() << endl;

    return 0;
}
```

Várias sobrecargas para um operador

- Sobrecarga de operador
  - Permite que se defina o comportamento de operadores comuns
- Sobrecarga de função
  - Permite repetir nomes de funções
- Sobrecarga da função da sobrecarga do operador
  - Suporte para aquela operação para diferentes tipos de parâmetros

Várias sobrecargas para um operador

- Queremos manipular a classe **Inteiro**
- Método especial **operator+**
  - Adição de dois objetos **Inteiro**
  - Adição de um objeto **Inteiro** com um valor
  - Adição de um valor com um objeto **Inteiro**
    - A ordem importa!

```
class Inteiro{
private:
    int valor;

public:
    Inteiro(int _valor): valor {_valor}{ }
    // Declaração a sobrecarga de operador como "friend"
    friend Inteiro operator+(const Inteiro& v1, const Inteiro& v2);
    friend Inteiro operator+(const Inteiro& v, const int valor);
    friend Inteiro operator+(const int valor, const Inteiro& v);

    int getInteiro() const { return valor; }
};

Inteiro operator+(const Inteiro& v1, const Inteiro& v2){
    return Inteiro(v1.valor + v2.valor);
}

Inteiro operator+(const Inteiro& v, const int valor){
    return Inteiro(v.valor + valor);
}

Inteiro operator+(const int valor, const Inteiro& v){
    return Inteiro(v.valor + valor);
}
```

Várias sobrecargas para um operador

- O compilador seleciona qual implementação da função deverá ser utilizada com base nos operandos passados
  - objeto + objeto
  - objeto + valor
  - valor + objeto

```
int main(){
    Inteiro inteiro1(6);
    Inteiro inteiro2 = inteiro1 + 5;
    Inteiro inteiro3 = 10 + inteiro1;
    Inteiro inteiro4 = inteiro1 + inteiro2;

    cout << "Valor: " << inteiro1.getInteiro() << endl;
    cout << "Valor: " << inteiro2.getInteiro() << endl;
    cout << "Valor: " << inteiro3.getInteiro() << endl;
    cout << "Valor: " << inteiro4.getInteiro() << endl;

    return 0;
}
```

Sobrecarga da leitura e escrita

- Atributos privados
  - Mais seguros, mas torna o processo de leitura/escrita de um objeto uma tarefa árdua

```
int main(){
    Inteiro inteiro1(6);
    Inteiro inteiro2 = inteiro1 + 5;
    Inteiro inteiro3 = 10 + inteiro1;
    Inteiro inteiro4 = inteiro1 + inteiro2;

    cout << "Valor: " << inteiro1.getInteiro() << endl;
    cout << "Valor: " << inteiro2.getInteiro() << endl;
    cout << "Valor: " << inteiro3.getInteiro() << endl;
    cout << "Valor: " << inteiro4.getInteiro() << endl;

    return 0;
}
```

Sobrecarga da leitura e escrita

- Atributos privados
  - É preciso acessar os métodos definidos para cada atributo para fazer a sua leitura
  - Na impressão, podemos criar uma função que realiza toda a escrita formatada.

```
class Ponto {
private:
    int x;
    int y;
public:
    Ponto(int _x = 0, int _y = 0): x {_x}, y {_y}{ }
    void setX(int _x){ x = _x;}
    void setY(int _y){ y = _y;}

    void exibe(){
        cout << "Ponto(" << x << ", " << y << ")" << endl;
    }
};

int main() {
    int x, y;
    cout << "Digite um ponto: ";
    cin >> x >> y;

    Ponto p1(x,y);
    Ponto p2(4, 6);

    p1.exibe();
    p2.exibe();

    return 0;
}
```

Sobrecarga da leitura e escrita

- Sobrecarga do operador de extração **>>**
  - Podemos fazer com que os dados sejam armazenados diretamente nos atributos da classe
- Sobrecarga do operador de inserção **<<**
  - Acessa diretamente os atributos do objeto e faz a impressão

```
class Ponto {
private:
    int x;
    int y;
public:
    Ponto(int _x = 0, int _y = 0): x {_x}, y {_y}{ }

    friend ostream& operator<< (ostream& out, const Ponto& p);
    friend istream& operator>> (istream& in, Ponto& p);
};

ostream& operator<< (ostream& out, const Ponto& p){
    out << "Ponto(" << p.x << ", " << p.y << ")" << endl;
    return out;
}

istream& operator>> (istream& in, Ponto& p){
    in >> p.x;
    in >> p.y;
    return in;
}

int main() {
    Ponto p1;
    cout << "Digite um ponto: ";
    cin >> p1;

    Ponto p2(4, 6);
    cout << p1 << p2 << endl;

    return 0;
}
```

Sobrecarga de incremento/decremento

- Funciona como a sobrecarga de outros operadores matemáticos
- Porém, precisa retornar uma referência ao objeto atual
  - Por exemplo, em

```
x = a + b++
```

  - a variável b é incrementada e usada em outra opção aritmética

Ponteiro this | Relembrando...

- Ponteiro especial presente em todas as classes em C++
  - Está disponível dentro dos métodos não estáticos de uma classe
  - Utilizado para referenciar o próprio objeto que invocou o método
  - Permite acesso direto aos seus membros
  - Facilita diversas operações na programação orientada a objetos

Sobrecarga de incremento/decremento

- Sobrecarga de função combinada com a sobrecarga de operador
  - Faz uso do ponteiro **this**
  - Precisa retornar o objeto antes ou depois da operação de incremento/decremento
    - Assim permite o encadeamento com outras operações

```
class Contador {
private:
    int valor;
public:
    Contador(int v = 0): valor {v}{ }

    Contador& operator++() {// Pré incremento (++valor)
        ++valor;
        return *this;
    }
    Contador operator++(int) {// Pós incremento (valor++)
        Contador temp = *this;
        ++valor;
        return temp;
    }
    Contador& operator--() {// Pré decremento (--valor)
        --valor;
        return *this;
    }
    Contador operator--(int) {// Pós decremento (valor--)
        Contador temp = *this;
        --valor;
        return temp;
    }

    friend ostream& operator<< (ostream& out, const Contador& c);
};

ostream& operator<< (ostream& out, const Contador& c){
    out << c.valor << endl;
    return out;
}
```

Sobrecarga de incremento/decremento

- Prefixado
  - Incrementa/decrementa o valor
  - Retorna uma referência ao objeto atual
- Pós-fixado
  - Salva o estado atual do objeto em uma variável temporária
  - Incrementa/decrementa o valor
  - Retorna o estado antigo armazenado na variável temporária

```
class Contador {
private:
    int valor;
public:
    Contador(int v = 0): valor {v}{ }

    Contador& operator++() {// Pré incremento (++valor)
        ++valor;
        return *this;
    }
    Contador operator++(int) {// Pós incremento (valor++)
        Contador temp = *this;
        ++valor;
        return temp;
    }
    Contador& operator--() {// Pré decremento (--valor)
        --valor;
        return *this;
    }
    Contador operator--(int) {// Pós decremento (valor--)
        Contador temp = *this;
        --valor;
        return temp;
    }

    friend ostream& operator<< (ostream& out, const Contador& c);
};

ostream& operator<< (ostream& out, const Contador& c){
    out << c.valor << endl;
    return out;
}
```

Sobrecarga de incremento/decremento

- 4 implementações
  - Contador& operator++()
    - incremento prefixado ++obj
  - Contador operator++(int)
    - incremento pós-fixado obj++
  - Contador& operator--()
    - decremento prefixado –obj
  - Contador operator--(int)
    - decremento pós-fixado obj--

```
class Contador {
private:
    int valor;
public:
    Contador(int v = 0): valor {v}{ }

    Contador& operator++() {// Pré incremento (++valor)
        ++valor;
        return *this;
    }
    Contador operator++(int) {// Pós incremento (valor++)
        Contador temp = *this;
        ++valor;
        return temp;
    }
    Contador& operator--() {// Pré decremento (--valor)
        --valor;
        return *this;
    }
    Contador operator--(int) {// Pós decremento (valor--)
        Contador temp = *this;
        --valor;
        return temp;
    }

    friend ostream& operator<< (ostream& out, const Contador& c);
};

ostream& operator<< (ostream& out, const Contador& c){
    out << c.valor << endl;
    return out;
}
```

Sobrecarga de operadores

- Isso permite manipular o valor de um objeto de maneira intuitiva
- Mais consistente com a semântica dos operadores padrão do C++

```
int main() {
    Contador c(5);
    cout << "Valor inicial: " << c;

    ++c;
    cout << "Após (++c): " << c;
    c++;
    cout << "Após (c++): " << c;
    --c;
    cout << "Após (--c): " << c;
    c--;
    cout << "Após (c--): " << c;

    return 0;
}
```

Material Complementar

- Aula 32 - POO: Sobrecarga de Operadores
  - https://youtu.be/qYsZFdwVWQo